

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium) Assessment

Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand

7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember

Code of Conduct:

*I **KARNAM HARSHITH /192511384** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure

Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1) CONCEPT MAP ON DBMS , SCHEMA , ER DIAGRAM , ATTRIBUTE

DBMS

- DataBase Management System
- Software used to define, store, manipulate & control access to data
- Acts as interface between user & database
- eg: MySQL, Oracle, PostgreSQL, SQL Server

DBMS

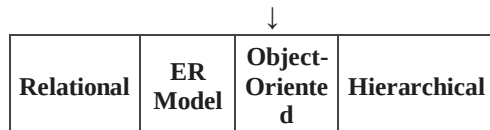


1) DATA MODEL

- Blueprint / concept used to organise data elements
- Standardises how data relates to other data
- Describes structure & constraints of a database

Types of Data Model

Data Model

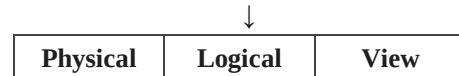


SCHEMA

- Overall logical design / structure of a database
- Defined at the time of database design
- Does NOT change frequently (blueprint of DB)

Types of Schema

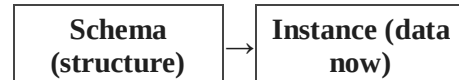
Schema



2) INSTANCE

- Data stored in the database at a particular moment
- Snapshot / current state of the database
- Changes frequently — every insert, update, delete

Schema vs Instance



- *Schema is fixed → Instance keeps changing over time*

ER DIAGRAM

- Entity - Relationship diagram
- Visual / graphical representation of entities, attributes & relationships
- Used for conceptual design of a database

Basic ER Notation



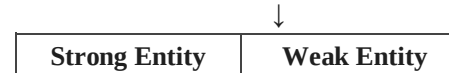
- *Oval = Attribute Rectangle = Entity Diamond = Relationship*

3) ENTITY

- A real world object / person / place with independent existence
- eg: Student, Employee, Car, Department
- Represented by a rectangle

Types of Entity

Entity

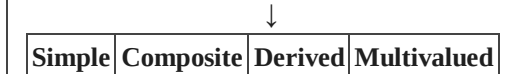


ATTRIBUTE

- A property / characteristic that describes an entity
- Represented by an oval / ellipse
- eg: name, age, id, address

Types of Attribute

Attribute

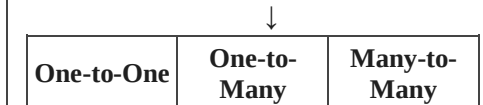


4) RELATIONSHIP

- An association between two or more entities
- Represented by a diamond shape
- eg: Teacher — teaches — Student

Cardinality of Relationship

Relationship

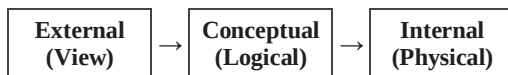


2) ANSI/SPARC THREE-LEVEL ARCHITECTURE — DATA INDEPENDENCE AT EACH

ANSI/SPARC

- Proposed in 1975 by the ANSI/SPARC Study Group
- Splits a database system into 3 levels of abstraction
- Separates how a user sees data from how it is physically stored
 - Main goal: achieve DATA INDEPENDENCE

The Three Levels



- Each level is linked to the next by a MAPPING

TWO MAPPINGS

- External / Conceptual mapping
- Conceptual / Internal mapping
- These mappings are what make data independence possible

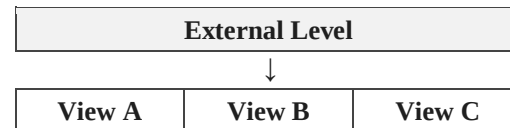
Purpose of the Architecture

- Hides physical storage complexity from users
- Supports many different user views of the same data
- Lets structure change without breaking applications

EXTERNAL LEVEL

- Highest level — the VIEW level, closest to the users
- Describes only the part of the database relevant to one user / application
- Many external views can exist — one per user group
- Defined using a "view schema"

Multiple User Views



- eg: Student view sees marks only; Accounts view sees fee details only

LOGICAL DATA INDEPENDENCE

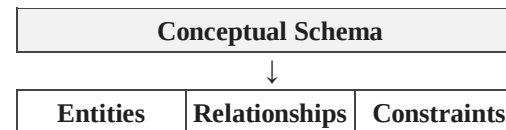
- Ability to change the CONCEPTUAL schema without changing external views or application programs
- eg: adding a new table/attribute to the DB does not disturb existing views

Conceptual → External (shielded by mapping)

CONCEPTUAL LEVEL

- Middle level — the LOGICAL / community level
- Describes WHAT data is stored & the relationships among data
- Represents entire database for the whole organisation
- Hides physical storage details, but hides user-specific views too
- Usually modelled using the ER Model

Conceptual Schema Contains



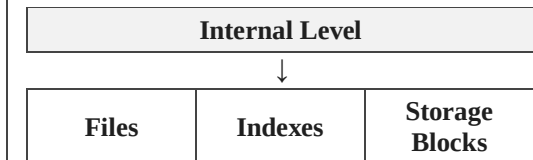
ROLE OF THIS LEVEL

- Acts as a BRIDGE between the external level (users) and internal level (storage)
- Only ONE conceptual schema per database
- Represents the DBA's / whole-enterprise view of data
- It is independent of hardware & storage structure

INTERNAL LEVEL

- Lowest level — the PHYSICAL level, closest to storage
- Describes HOW data is actually stored on disk
- Deals with data structures, file organisation & access paths
- Concerned with indexes, compression, encryption, storage allocation

Internal Schema Deals With



PHYSICAL DATA INDEPENDENCE

- Ability to change the INTERNAL schema (storage structure) without changing the conceptual schema
- eg: changing file organisation or adding an index does not affect the conceptual level or apps

Internal → Conceptual (shielded by mapping)

3) FILE-BASED SYSTEM VS DATABASE SYSTEM — PROBLEMS SOLVED BY DBMS

FILE-BASED SYSTEM

- Collection of application programs that provide services to end users
- Each application defines & manages its OWN data files independently
- No central control over the data
- The traditional approach used before DBMS existed

Separate Files per Application

File-Based System



Payroll File	Inventory File	Sales File
--------------	----------------	------------

CHARACTERISTICS

- Data & programs are tightly coupled
- Every application writes its own file-handling code
- Adding a new application means duplicating data all over again
- Different departments keep their own separate copies of data

These limitations led to the...
DATABASE SYSTEM

PROBLEMS FACED

Main Problems in File Systems

File-Based Problems



Redundancy	Isolation	Integrity	Security
------------	-----------	-----------	----------

1) DATA REDUNDANCY & INCONSISTENCY

- Same data is duplicated across many files
- Wastes storage & causes mismatched values

2) DATA ISOLATION

- Data is scattered across files in different formats
- Hard to write new programs to retrieve the right data

3) INTEGRITY PROBLEMS

- Constraints are buried inside program code
- Hard to enforce or change consistently

4) ATOMICITY, CONCURRENCY & SECURITY

- A crash can leave a file half-updated (no atomicity)
- Simultaneous access by many users causes anomalies
- No fine-grained access control on plain files

DATABASE SYSTEM

- Database = shared collection of logically related data
- DBMS = software that defines, creates, maintains & controls access to the database
- Centralises data into ONE repository used by every application
- Data is independent of the application programs that use it

All Applications Share One Database

Shared Database (via DBMS)



Payroll App	Inventory App	Sales App
-------------	---------------	-----------

KEY IDEA

- Each fact is stored once, defined once, and used by many applications
- DBMS acts as a controlled gatekeeper between apps & the data
- Structure of data is described once — in the schema — not in every program

DBMS SOLUTIONS

- DBMS was built specifically to remove the weaknesses of file systems

REDUNDANCY → CONTROLLED

- Centralised data means each fact is stored only once

ISOLATION → INTEGRATED

- One database integrates all related data for easy, flexible retrieval

INTEGRITY → ENFORCED

- Constraints defined once in the schema, enforced automatically by DBMS

ATOMICITY → GUARANTEED

- DBMS uses TRANSACTIONS — operations complete fully or not at all

CONCURRENCY → MANAGED

- DBMS uses concurrency control (locking) for safe simultaneous access

SECURITY → ENFORCED

- DBMS provides authorisation & user-level access control

RELATIONSHIP: EACH FILE-SYSTEM PROBLEM → THE DBMS SOLUTION THAT FIXES IT

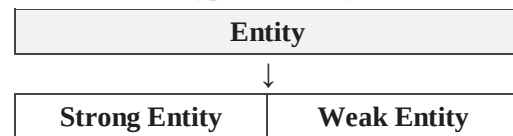
PROBLEM IN FILE-BASED SYSTEM	RELATIONSHIP	HOW DBMS SOLVES IT
Data Redundancy & Inconsistency	→ solved by →	Centralised storage — each fact stored ONCE, shared by all apps
Data Isolation	→ solved by →	Integrated database — related data linked & easily retrievable
Integrity Problems	→ solved by →	Constraints defined once in the schema & enforced automatically
Atomicity Problems	→ solved by →	Transactions guarantee all-or-nothing execution
Concurrent Access Anomalies	→ solved by →	Concurrency control (locking / scheduling) manages simultaneous access
Security Problems	→ solved by →	Authorisation & user-level access control built into the DBMS

4) ER DIAGRAM COMPONENTS — ENTITIES, ATTRIBUTES, RELATIONSHIPS & PARTICIPATION

ENTITY

- A real world object / person / place with independent existence
- Represented by a RECTANGLE
- eg: Student, Employee, Car, Department

Types of Entity



STRONG ENTITY

- Has its own primary key attribute
- Exists independently of any other entity
- Drawn as a single rectangle

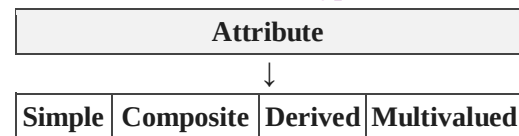
WEAK ENTITY

- Does NOT have enough attributes to form its own primary key
- Depends on an OWNER / identifying entity for existence
- Has a PARTIAL KEY (discriminator), underlined with a dashed line
- Drawn as a DOUBLE rectangle; linked by a double diamond
- eg: "Installment" depends on "Loan"; "Dependent" depends on "Employee"

ATTRIBUTE TYPES

- A property / characteristic that describes an entity
- Represented by an OVAL

Four Main Types



1) SIMPLE ATTRIBUTE

- Cannot be divided into smaller parts
- eg: age, id, roll number

2) COMPOSITE ATTRIBUTE

- Made up of several smaller sub-attributes
- eg: Name → First, Middle, Last

3) DERIVED ATTRIBUTE

- Value is derived from another stored attribute
- Drawn with a DASHED oval
- eg: Age is derived from Date of Birth

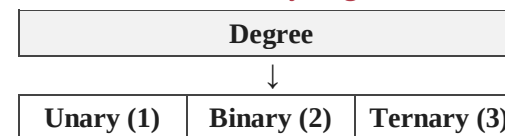
4) MULTIVALUED ATTRIBUTE

- Can hold more than one value at a time
- Drawn with a DOUBLE oval
- eg: a person can have several phone numbers

RELATIONSHIP TYPES

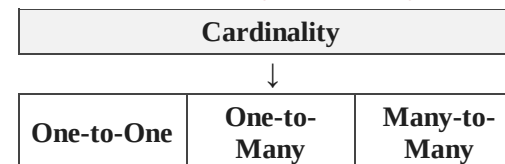
- An association between two or more entities
- Represented by a DIAMOND
- eg: Teacher — teaches — Student

Classified by Degree



- *Unary: relates an entity to itself — eg: Employee supervises Employee*
- *Binary: relates two entities — the most common type*
- *Ternary: relates three entities together*

Classified by Cardinality

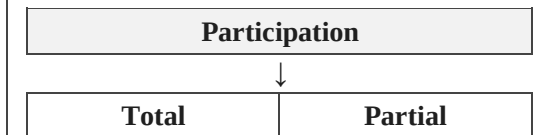


- *Cardinality = how many instances of one entity relate to instances of another*

PARTICIPATION

- States whether ALL or only SOME entity instances take part in a relationship

Two Types



TOTAL PARTICIPATION

- EVERY instance of the entity MUST take part in the relationship
- Drawn using a DOUBLE line connecting entity & relationship
- eg: every Employee must work in some Department

PARTIAL PARTICIPATION

- SOME instances may or may not take part in the relationship
- Drawn using a SINGLE line connecting entity & relationship
- eg: not every Employee manages a Department

QUICK EXAMPLE



ER DIAGRAM NOTATION — QUICK REFERENCE LEGEND

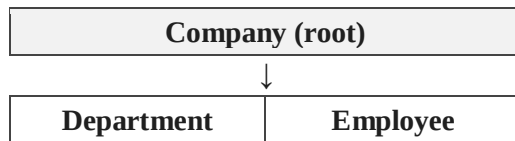
SYMBOL	COMPONENT	MEANING
□	Entity	A real-world object with independent existence
□□	Weak Entity	Depends on an owner entity; has only a partial key
○	Attribute (simple)	A basic property describing an entity
○○	Multivalued Attribute	Can take more than one value
- - ○ - -	Derived Attribute	Value is derived from another attribute
◇	Relationship	Association linking two or more entities
◇◇	Identifying Relationship	Connects a weak entity to its owner entity
—	Partial Participation	Some entity instances may not take part
==	Total Participation	Every entity instance MUST take part

5) DATA MODELS COMPARED — HIERARCHICAL, NETWORK & RELATIONAL

HIERARCHICAL MODEL

- Organises data in a TREE-like structure of records
- Built on PARENT–CHILD relationships
- Each child record has exactly ONE parent
- One of the earliest data models (1960s)

Tree Structure



KEY PROPERTIES

- Supports ONE-TO-MANY relationships only
- Traversal always starts at the ROOT, moving top-down
- Fast for predefined, repetitive queries
- Very rigid — hard to represent many-to-many links

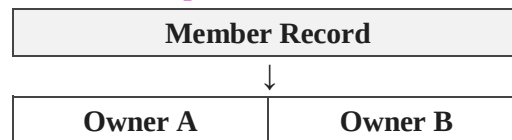
EXAMPLE SYSTEMS

- IBM's IMS (Information Management System)
- *Windows Registry, XML document structure*

NETWORK MODEL

- Extension of the hierarchical model
- Records connected as a GRAPH using OWNER–MEMBER "sets"
- A child (member) CAN have MULTIPLE parents (owners)
- Standardised by CODASYL in the 1970s

Graph / Set Structure



KEY PROPERTIES

- Supports MANY-TO-MANY relationships
- More flexible than the hierarchical model
- Navigation is via pointers — complex to design & maintain
- No single fixed entry point like the hierarchical root

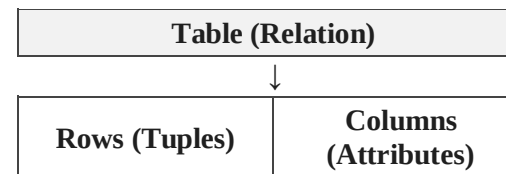
EXAMPLE SYSTEMS

- IDMS (Integrated Database Management System)
- *IDS (Integrated Data Store)*

RELATIONAL MODEL

- Organises data into TABLES (relations) of rows & columns
- Based on set theory & predicate logic — proposed by E.F. Codd (1970)
- Relationships are built using FOREIGN KEYS, not physical pointers

Table Structure



KEY PROPERTIES

- Strong logical & physical DATA INDEPENDENCE
- Flexible querying using SQL (non-navigational)
- NORMALIZATION reduces redundancy & anomalies
- Easiest model to understand & restructure

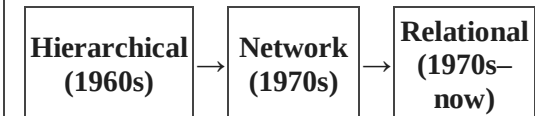
EXAMPLE SYSTEMS

- MySQL, Oracle, PostgreSQL, SQL Server

EVOLUTION

- Data models evolved to handle growing data complexity

From Rigid to Flexible



LIMITATIONS OF OLDER MODELS

- Hierarchical: rigid; cannot represent many-to-many directly
- Network: flexible, but complex pointer-based navigation

ADVANTAGES OF THE RELATIONAL MODEL

- Simple, intuitive table structure
- SQL gives powerful, declarative querying
- Schema can change without breaking applications
- Normalization keeps redundancy low

RESULT

- The relational model became the dominant choice in modern DBMS

SIDE-BY-SIDE COMPARISON OF KEY PROPERT

PROPERTY	HIERARCHICAL	NETWORK	RELATIONAL
Structure	Tree — parent/child records	Graph — owner/member sets	Tables — rows & columns
Relationship Type	One-to-Many only	Many-to-Many supported	Any type (via foreign keys)
Data Access	Navigational, top-down from root	Navigational, via pointers/sets	Declarative (SQL) — non-navigational
Flexibility	Low — rigid structure	Medium — complex to modify	High — easy to restructure
Data Redundancy	Can be high	Can be high	Reduced via normalization
Example DBMS	IBM IMS	IDMS, IDS	MySQL, Oracle, PostgreSQL

6) ENHANCED ER (EER) MODEL — CONCEPT MAP

ER MODEL → EER MODEL

- Basic ER Model represents entities, attributes & relationships only
- Cannot express hierarchies, class/subclass structures, or part-whole relationships
- Enhanced ER (EER) Model adds semantic modelling power on top of ER

Extension Path

Basic ER Model

↓ *adds subclass / superclass concepts*

Enhanced ER (EER) Model

NEW CONCEPTS INTRODUCED

- Generalization — combine similar entities bottom-up
- Specialization — split an entity top-down
- Aggregation — model a whole-part relationship
- Union / Category — subclass with multiple superclasses

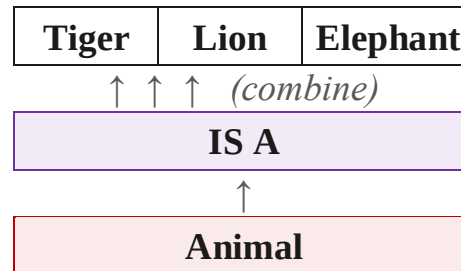
WHY EER IS NEEDED

- Real-world data has natural hierarchies (e.g. Vehicle → Car, Truck)
- Improves semantic accuracy of the database design

GENERALIZATION

- Bottom-up approach
- Combines multiple entity types that share common attributes into one general superclass
- The superclass holds properties common to all the generalized entities

Example



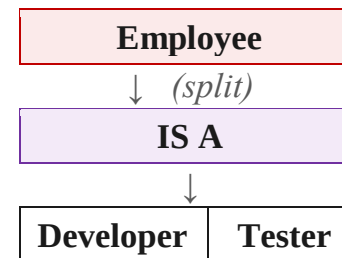
KEY POINTS

- Minimizes redundant attribute definitions
- Superclass = union of its subclasses
- Also called Union / Category when a subclass has more than one superclass

SPECIALIZATION

- Top-down approach
- Identifies distinct sub-groupings within an entity type
- Each subclass adds attributes / relationships specific only to itself

Example



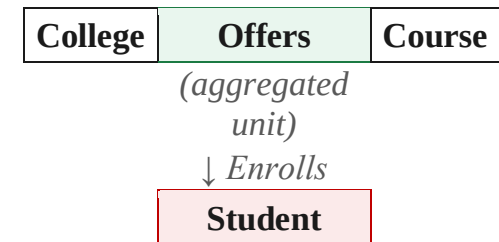
CONSTRAINTS

- Disjoint — an entity belongs to only one subclass
- Overlapping — an entity may belong to more than one subclass
- Total — every superclass entity must belong to a subclass
- Partial — a superclass entity need not belong to any subclass

AGGREGATION

- Represents a whole-part ("has-a") relationship
- Models a relationship set as a higher-level entity so it can take part in another relationship

Example



EER vs ER — QUICK RECAP

- Generalization → bottom-up, combine entities into a superclass
- Specialization → top-down, split an entity into subclasses
- Aggregation → whole-part relationship treated as one unit
- Category/Union → subclass of multiple superclasses

RESULT

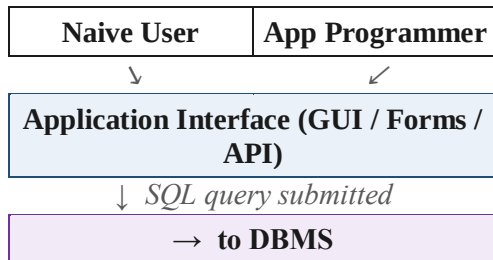
- EER lets the database capture real-world hierarchies and part-whole structures that plain ER cannot

7) DBMS SOFTWARE ARCHITECTURE — CONCEPT MAP

USER / APPLICATION LAYER

- Entry point where a request for data originates
- Users interact through GUIs, forms, or application programs
- Request is expressed as an SQL query or an API call

Request Flow



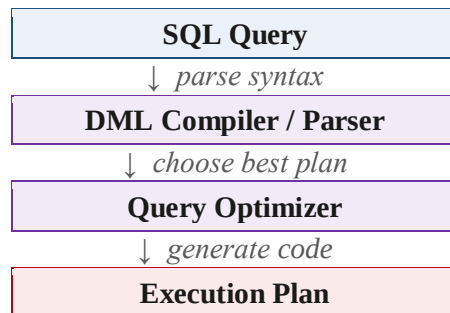
USER TYPES

- Naive users — use pre-built forms / apps
- Application programmers — write programs using APIs (e.g. JDBC/ODBC)
- Sophisticated users — write SQL queries directly
- Database administrators (DBA) — manage the whole system

QUERY PROCESSOR

- Receives the query and converts it into low-level instructions
- Checks syntax, resolves table/column names, and plans the most efficient execution

Processing Flow



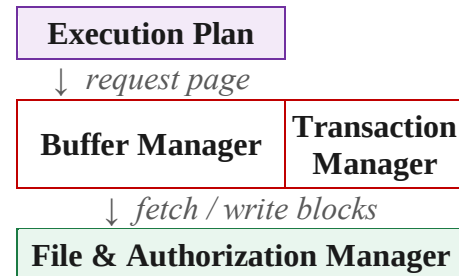
KEY COMPONENTS

- DDL interpreter — processes schema definitions
- DML compiler — translates DML statements into low-level instructions
- Query optimizer — picks the lowest-cost access path (indexes, join order)
- Embedded DML precompiler — handles queries inside host-language code

STORAGE MANAGER

- Executes the plan — the bridge between the query processor and physical data
- Ensures data consistency, concurrency, and recovery while accessing storage

Execution Flow



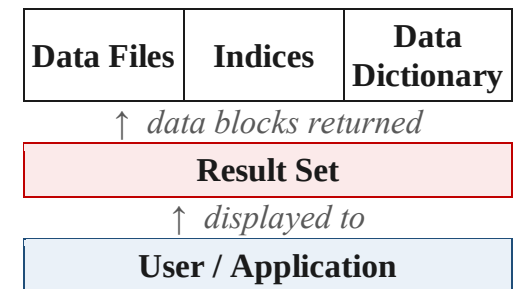
KEY COMPONENTS

- Buffer manager — brings needed data pages from disk into memory
- Transaction manager — enforces ACID properties & concurrency control
- File manager — manages allocation of disk space & file structures
- Authorization & integrity manager — checks access rights & constraints

DISK STORAGE & RETRIEVAL

- Physical layer where data actually resides
- Requested blocks are read, and results travel back up to the user

Retrieval Flow



PHYSICAL STRUCTURES

- Data files — store the actual table records
- Indices — speed up lookups without scanning entire files
- Data dictionary — stores metadata (schema, constraints, statistics)

END-TO-END FLOW

- User Request → Query Processor → Storage Manager → Disk → Result back to User

8) CO1 DATA MODELS → REAL-WORLD APPLICATIONS & SUITABILITY

HIERARCHICAL MODEL

- Tree-structured parent–child records; each child has exactly one parent

Application Mapping

File Directory System

↓ *natural parent-child paths*

XML / Windows Registry

REAL-WORLD APPLICATIONS

- File systems — folders containing sub-folders and files
- XML document storage — nested tag hierarchies
- Windows Registry — settings organized as nested keys
- Organizational charts — single reporting manager per employee

SUITABILITY

- Best when data is naturally tree-shaped with a fixed 1:N structure
- Fast, predictable traversal from a single root
- Poor fit when a record needs multiple parents (many-to-many)

NETWORK MODEL

- Graph-structured owner–member sets; a record can have multiple owners

Application Mapping

Bill of Materials (BOM)

↓ *a part used in many products*

Airline Reservation Systems

REAL-WORLD APPLICATIONS

- Manufacturing Bill of Materials — a part belongs to many products
- Legacy airline / railway reservation systems
- Telecom network routing databases (pre-relational era)
- Early CODASYL-based banking systems

SUITABILITY

- Best when records genuinely need many-to-many links
- More flexible than hierarchical, but pointer navigation is complex
- Largely replaced today by the relational model with foreign keys

RELATIONAL MODEL

- Table-based structure; relationships via foreign keys, queried with SQL

Application Mapping

Banking & Finance Systems

↓ *needs strict consistency (ACID)*

E-commerce, ERP, Payroll

REAL-WORLD APPLICATIONS

- Banking systems — accounts, transactions, ledgers
- E-commerce platforms — orders, customers, inventory
- ERP / payroll / student information systems
- Almost all modern general-purpose apps (MySQL, Oracle, PostgreSQL)

SUITABILITY

- Best for structured, tabular data needing strong consistency
- SQL gives flexible, ad-hoc querying not tied to a fixed path
- Normalization keeps redundancy low — ideal for transactional data

OBJECT-ORIENTED MODEL

- Stores data as objects — combines data & behaviour, supports inheritance

Application Mapping

CAD / CAM Design Systems

↓ *complex nested objects*

Multimedia & GIS Databases

REAL-WORLD APPLICATIONS

- CAD/CAM — engineering parts modeled as reusable objects
- Multimedia databases — audio, video, image objects
- Geographic Information Systems (GIS) — spatial objects
- Object-oriented software with persistent class instances

SUITABILITY

- Best when data is complex, nested, or behaviour-rich
- Inheritance models real-world class hierarchies naturally
- Less suited to simple tabular, high-volume transactional data

QUICK RECAP

- Choice of model depends on data shape, relationship complexity & consistency needs

9) SCHEMAS, INSTANCES & DATA INDEPENDENCE — CONCEPT MAP

SCHEMA

- Overall design / structure of a database — defined at DB design time
- Rarely changes once the database is created
- Described using a Data Definition Language (DDL)

Three-Schema Architecture

External Schema (views)



Conceptual Schema (logical)



Internal Schema (physical)

THE THREE LEVELS

- External — how individual users/apps view the data
- Conceptual — the community/logical view (entities, relationships, constraints)
- Internal — physical storage: files, indexes, access paths

INSTANCE

- The actual data stored in the database at a particular moment in time
- Also called the database state, or a "snapshot"
- Changes constantly with every insert, update, or delete

Schema vs Instance

Schema
(structure)

+ Data



Instance (snapshot at time t)

KEY DISTINCTION

- Schema = the blueprint; Instance = the current content
- A single schema can have many different instances over time
- Analogy: schema is a table's column layout; instance is today's rows

LOGICAL DATA INDEPENDENCE

- Ability to change the conceptual schema without changing external schemas or application programs
- Protects applications from changes in logical structure

What Can Change

Conceptual Schema changes

× isolated from ↑

External Schema / Apps
unaffected

EXAMPLES

- Adding a new attribute or table to the conceptual schema
- Splitting/merging tables without breaking existing views

DIFFICULTY

- Harder to achieve than physical independence — apps are tied more closely to logical structure

PHYSICAL DATA INDEPENDENCE

- Ability to change the internal schema without changing the conceptual schema
- Protects the logical structure from changes in physical storage

What Can Change

Internal Schema changes

× isolated from ↑

Conceptual Schema unaffected

EXAMPLES

- Adding a new index or changing file organization
- Moving data to different storage devices, compressing files

DIFFICULTY

- Easier to achieve — physical storage details are hidden below the conceptual level

RELATIONSHIP RECAP

- Three-schema architecture + separation of levels is what makes both types of data independence possible

10) ROLES & RESPONSIBILITIES IN A DBMS ENVIRONMENT — CONCEPT MAP

DATABASE ADMINISTRATOR (DBA)

- Central authority who controls the entire database system
- Has the highest level of technical & administrative privileges

Role Flow



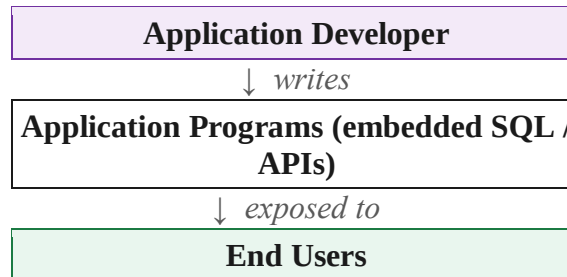
RESPONSIBILITIES

- Schema definition — creates the original database schema (DDL)
- Storage & access method definition — decides physical structures & indexes
- Granting authorization — controls who can access/modify what data
- Routine maintenance — backup, recovery, performance tuning, monitoring
- Ensures data security, integrity, and concurrency control policies

APPLICATION DEVELOPER

- Builds programs that let end users interact with the database
- Works within the schema & constraints defined by the DBA

Role Flow



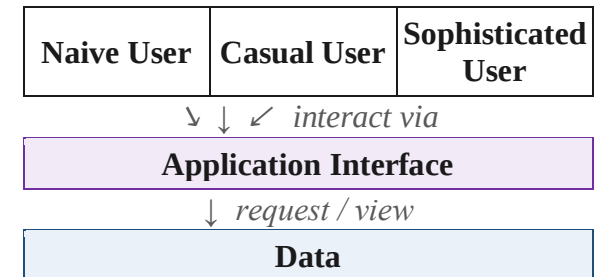
RESPONSIBILITIES

- Writes application programs using DML embedded in a host language, or via APIs (JDBC/ODBC)
- Designs forms, reports, and user interfaces for end users
- Implements business logic and validates data at the application layer
- Tests and debugs interactions between the app and the DBMS
- Coordinates with the DBA on schema changes affecting the application

END USER

- Consumes the database through applications, without needing internal details
- Typically has no knowledge of the database's physical structure

Role Flow



USER TYPES

- Naive users — interact via pre-built forms/menus, e.g. bank tellers
- Casual users — occasionally query the DB using SQL, e.g. managers
- Sophisticated users — write complex queries/reports directly

HOW THE ROLES CONNECT

- DBA defines & secures the schema → Developer builds applications on it → End User consumes data through those applications

THANK YOU